

CSE233 Week 13:

Namespaces

Throughout the course, we've been putting `std::` in front of the standard library functions, types, and objects we've been using (e.g., `std::cout`, `std::string`, `std::getline()`). Let's discuss what exactly namespaces are and why they are important in C++.

Namespaces

To understand why namespaces are important, let's discuss how C/C++ programs are compiled. We've been using `#include` directives to include libraries into our source code. The C/C++ compilers are not extremely sophisticated. When encountering an `#include` directive, the compiler copies the code from that file into the source code file being compiled. Therefore, any time you use `#include <iostream>`, all of the code in the file `iostream` (and any code that is included by `iostream`) is copied into the source code being compiled. This means that a lot of unknown code may be dumped into a single file to be compiled any time we compile a program. Any variable, class, function, etc. names that are duplicated across these files can lead to a clash. We cannot assume that we know every name that is potentially used in an external file. That is why the C++ standard libraries all use the `std` namespace. This prevents code in the standard library files from clashing with users' code.

To demonstrate this on a smaller scale, let's consider the following code from three C++ files:

math

```
// a math/physics vector has x,y,z components
class Vector { /* ... */ };
```

datatypes

```
// a vector datatype is an automatically resizing dynamic array
class Vector { /* ... */ };
```

main.cpp

```
#include <datatypes>
#include <math>

int main() {
    // ??? which vector do we want ???
    Vector vec;
    // ...
    return 0;
}
```

For one, as the comment says, there is no way to know which vector we want. Also, when the preprocessor runs, the file that is being compiled will look something like this:

```
// a math/physics vector has x,y,z components
class Vector { /*...*/ };
// a vector datatype is an automatically resizing dynamic array
class Vector { /*...*/ };

int main() {
    // ??? which vector do we want ???
    Vector vec;
    // ...
    return 0;
}
```

Names cannot be reused like this, so the code will fail to compile because the name `Vector` has been redeclared, as far as the compiler is concerned. However, we can use namespaces to isolate the names from each other. A namespace is created by putting the `namespace` keyword followed by the name of the namespace and a block. Any code in the block is part of the namespace.

math

```
namespace math {
    // a math/physics vector has x,y,z components
    class Vector { /*...*/ };
}
```

datatypes

```
namespace datatypes {
    // a vector datatype is an automatically resizing dynamic array
    class Vector { /*...*/ };
}
```

main.cpp

```
#include <datatypes>
#include <math>

int main() {
    math::Vector vec;
    // ...
    return 0;
}
```

Now, the names are in separate namespaces, meaning we have a way to specify which version of `Vector` we want to use. It also ensures we are not risking a redeclaration when the files are combined during compilation. While this is a trivial example, when the code we are including is potentially thousands of lines long, using namespaces becomes critical to keep the global namespace clean.

Using Statements

A `using namespace` statement is used to bring all the elements of a namespace into the global namespace. This is not recommended practice in C++. `using namespace` statements can also be used within a function. This is better, since the using statement is contained to the function. However, I've found in practice that compilers (especially MSVC) decide to ignore using statements. Generally, `using namespace` statements are not used in C++. Single identifiers can be used as well:

```
int main() {
    using std::cout;
    cout << "hello\n";

    return 0;
}
```

Conclusion

When C++ programs are compiled, all the code that is included is combined into a single file. Namespaces allow us to keep the global namespace clean by keeping code isolated. While `using namespace` statements can be used to bring code from namespaces into the global namespace, this is considered bad practice in C++, and anecdotally I've found it to not work sometimes.